Ensemble Quantum Computing Laboratory

Prelab 4

Runzhao Guo

March 2, 2025

Problem 1

part a).

One-qubit X gate: $180I_x$
One-qubit H gate: $45I_y$ - $180I_x$ - $45I_{-y}$
CNOT: $180I_x$ - $90I_y$ - $\frac{1}{4J}$ - $90I_{-x}$ - $180I_x$
CZ(CNOT with Hadamard on both side of target qubit):$45S_y$ - $180S_x$ - $45S_{-y}$ - $180I_x$ - $90I_y$ - $\frac{1}{4J}$ - $90I_{-x}$ - $180I_x$ - $45S_y$ - $180S_x$ - $45S_{-y}$

part b).

The unitary evolution of pulse sequence for X gate on the first qubit is:

$$\begin{pmatrix} 0 & 0 & -i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & -i & 0 & 0 \end{pmatrix}$$

The unitary evolution of pulse sequence for H gate is:

$$-\frac{1}{\sqrt{2}}i\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}$$

The unitary evolution of pulse sequence for CNOT gate with the first qubit as control qubit and the second qubit as target qubit is:

$$\frac{1}{\sqrt{2}}\begin{pmatrix} 1-i & 0 & 0 & 0 \\ 0 & 1-i & 0 & 1 \\ 0 & 0 & 0 & 1-i \\ 0 & 0 & 1-i & 0 \end{pmatrix}$$

The unitary evolution of pulse sequence for CZ gate with the first qubit is the control qubit and the second qubit is the target qubit is:

$$-\frac{1}{\sqrt{2}}\begin{pmatrix} 1-i & 0 & 0 & 0 \\ 0 & 1-i & 0 & 1 \\ 0 & 0 & 1-i & 0 \\ 0 & 0 & 0 & -(1-i) \end{pmatrix}$$

These gates are correct up to a global phase, the main function that produce this result is shown in Appendix, program section.

Problem 2

Part 1

(a) The Four Classical Functions

Since x is a single bit, the possible output bits are either constants or else (possibly) depend on x. The four distinct classical functions are:

$$f_1(x) = 0, \quad f_2(x) = 1, \quad f_3(x) = x, \quad f_4(x) = \overline{x} \quad \text{(the NOT of } x\text{)}.$$

(b) Classical Circuit Implementations

- $\boldsymbol{f_1(x) = 0}$:

  A circuit that ignores the input and outputs '0' can be implemented by simply tying the output bit to logic-level 0 (ground) or using a constant-zero gate. In standard logic symbols:

  $$\text{Output} = 0.$$

- $\boldsymbol{f_2(x) = 1}$:

  Similarly, output is always 1, which can be implemented by connecting the output to a logic-level 1 (supply voltage). Symbolically:

  $$\text{Output} = 1.$$

- $\boldsymbol{f_3(x) = x}$:

  This is the *identity* function. A classical circuit is simply a *wire*; the output is just the same as the input:

  $$\text{Output} = x.$$

- $\boldsymbol{f_4(x) = \overline{x}}$:

  This is the *NOT* gate. A classical circuit is a standard inverter:

  $$\text{Output} = \overline{x}.$$

(c) Quantum Circuits with Rotations & CNOT

- For f_1 and f_2 (constant functions), we can implement them using single-qubit rotations that effectively flip the second qubit if necessary (to set the output to 1, or leave it at 0).
- For f_3 and f_4, we can use a CNOT from the first qubit (control) to the second qubit (target). If we want $|y \oplus x\rangle$, we apply a standard CNOT. For $|y \oplus \overline{x}\rangle$, we can apply an additional π-phase rotation or an X gate on the target prior to the CNOT, etc.

$$\begin{aligned}
&\text{(i) } f_1(x) = 0: \quad U_{f_1}(|x\rangle\,|y\rangle) = |x\rangle\ |\,y \oplus 0\rangle = |x\rangle\ |y\rangle\,.\\
&\text{(ii) } f_2(x) = 1: \quad U_{f_2}(|x\rangle\,|y\rangle) = |x\rangle\ |\,y \oplus 1\rangle = X(1)\,|x\rangle\ |y\rangle\,.\\
&\text{(iii) } f_3(x) = x: \quad U_{f_3}(|x\rangle\,|y\rangle) = |x\rangle\ |\,y \oplus x\rangle = \text{CNOT}(0,1)\ |x\rangle\,|y\rangle\,.\\
&\text{(iv) } f_4(x) = x: \quad U_{f_4}(|x\rangle\,|y\rangle) = |x\rangle\ |\,y \oplus 1 \oplus x\rangle = X(1)\text{CNOT}(0,1)\ |x\rangle\,|y\rangle\,.
\end{aligned}$$

Part 2: Pulse Sequences for Each f_k

We now specify four pulse sequences (labeled $U_{f_1}, U_{f_2}, U_{f_3}, U_{f_4}$) that implement the quantum circuits from Part 1

$f_1(x) = 0$

Goal: $|x\rangle\,|y\rangle \mapsto |x\rangle\ |y \oplus 0\rangle = |x\rangle\,|y\rangle$, i.e. the *identity* on two qubits.

Implementation:

Since the overall operation is simply the identity, an NMR pulse sequence can just do *nothing* (no pulses, no free evolution), or equivalently:

$$U_{f_1} \;=\; \hat{I} \quad \text{(no pulses, no delay)}.$$

$f_2(x) = 1$

Goal: $|x\rangle\,|y\rangle \mapsto |x\rangle\ |y \oplus 1\rangle = |x\rangle\ X\,|y\rangle\,.$

Implementation:

Apply a single-qubit π rotation on the *second* spin (qubit 1) about, say, the $\hat{x}$ axis:

$$U_{f_2} \;=\; R_x^{(1)}(\pi)$$

$f_3(x) = x$

Goal: $|x\rangle\,|y\rangle \mapsto |x\rangle\ |y \oplus x\rangle$, which is exactly a *CNOT* (control = qubit 0, target = qubit 1).

Implementation (typical NMR CNOT):

A standard way to realize CNOT using the Ising-like coupling is

$$U_{f_3} \;=\; R_y^{(0)}\!\left(\tfrac{\pi}{2}\right)\, U_{zz}\!\left(\tfrac{1}{4J}\right)\, R_x^{(0)}\!\left(-\tfrac{\pi}{2}\right)$$

where

$$U_{\mathrm{ZZ}}(\tau) \;=\; e^{-\,i\,2\pi J\,\tau\,\hat{I}_z^{(1)}\hat{I}_z^{(2)}} \quad \text{(free evolution for time } \tau\text{)}.$$

By choosing $\tau = 1/(4J)$, this sequence implements the usual controlled-NOT up to an overall phase, i.e. $|x\rangle|y\rangle \mapsto |x\rangle\,|x \oplus y\rangle$.

$f_4(x) = \overline{x}$

Goal: $|x\rangle\,|y\rangle \mapsto |x\rangle\ |\,y \oplus \overline{x}\rangle = |x\rangle\ |\,y \oplus 1 \oplus x\rangle\,.$

Implementation Idea:

We can realize $y \oplus \overline{x}$ as $y \oplus x \oplus 1$, i.e. do a CNOT *and* then flip the target qubit. Hence one simple pulse sequence is just

$$U_{f_4} \;= R_y^{(1)}\!\left(\tfrac{\pi}{4}\right)\, R_x^{(1)}(\pi)\, R_y^{(0)}\!\left(-\tfrac{\pi}{4}\right)\;\times\; U_{f_3} \;\times\; R_y^{(1)}\!\left(\tfrac{\pi}{4}\right)\, R_x^{(1)}(\pi)\, R_y^{(1)}\!\left(-\tfrac{\pi}{4}\right)$$

These four pulse sequences $U_{f_1}, U_{f_2}, U_{f_3}, U_{f_4}$ directly correspond to the four 2-qubit unitaries that implement the classical functions $f_k(x)$ as described in Part 1, but now in an *NMR pulse language* using (i) single-qubit $\hat{x}/\hat{y}$ rotations and (ii) a free-evolution (scalar-coupling) period $U_{\mathrm{ZZ}}(\tau)$.

Part 3:

$U_k = R_{\bar{y}2}(\frac{\pi}{2})R_{y1}(\frac{\pi}{2})U_{fk}R_{y2}(\frac{\pi}{2})R_{\bar{y}1}(\frac{\pi}{2})$
$k = 1$, $f_1(x) = 0$, $U_1 =$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$k = 2$, $f_2(x) = 1$, $U_2 =$

$$i\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

$k = 3$, $f_3(x) = x$, $U_3 =$

$$0.35\begin{pmatrix} -1+i & 1-i & 1+i & 1+i \\ -1-i & -1-i & -1+i & 1-i \\ 1+i & 1+i & -1+i & 1-i \\ -1-i & 1-i & -1-i & -1-i \end{pmatrix}$$

$k = 4$, $f_4(x) = \bar{x}$, $U_4 =$

$$0.35\begin{pmatrix} -1-i & -1-i & -1+i & 1-i \\ 1-i & -1+i & -1-i & -1-i \\ -1+i & 1-i & -1-i & -1-i \\ -1-i & -1-i & 1-i & -1+i \end{pmatrix}$$

Problem 3

The density matrix for final state averaged for three permutation matrices for f_1 is as follows:

$$\begin{pmatrix} 0.67 & 0.33 & 0 & 0 \\ 0.33 & 0.67 & 0 & 0 \\ 0 & 0 & 0.33 & 0 \\ 0 & 0 & 0 & 0.33 \end{pmatrix}$$

The density matrix for final state averaged for three permutation matrices for f_2 is as follows:

$$\begin{pmatrix} 0.67 & 0.33 & 0 & 0 \\ 0.33 & 0.67 & 0 & 0 \\ 0 & 0 & 0.33 & 0 \\ 0 & 0 & 0 & 0.33 \end{pmatrix}$$

The density matrix for final state averaged for three permutation matrices for f_3 is as follows:

$$\begin{pmatrix} 0.5 & 0.167i & 0.167i & -0.167 \\ -0.167i & 0.5 & 0.167 & 0.167i \\ -0.167i & 0.167 & 0.5 & 0.167i \\ -0.167 & -0.167i & -0.167i & 0.5 \end{pmatrix}$$

The density matrix for final state averaged for three permutation matrices for f_4 is as follows:

$$\begin{pmatrix} 0.5 & 0.167i & 0.167i & -0.167 \\ -0.167i & 0.5 & 0.167 & 0.167i \\ -0.167i & 0.167 & 0.5 & 0.167i \\ -0.167 & -0.167i & -0.167i & 0.5 \end{pmatrix}$$

The output for constant function is $|0\rangle$ and output for balanced function is $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, these indecate my simulation of two-qubit Deutsche-Jozsa Algorithm is correct. The complete pulse sequence along with a simple explaination of the syntax used is in Appendix, section Sequence.

Problem 4

How ASP is used to prepare the system qubit in the ground state?

The system qubit (^{13}C in choloroform) is initially prepared in the ground state of Hamiltonian $H_0 = -\sigma x$ via pseudo-Hadamard gate.

The system Hamiltonian is gradually changed from H_0 to the target molecular Hamiltonian H using the interpolation:

$$H_{ad} = (1 - s)H_0 + sH$$

, where $s = \frac{t}{T}$ and t grows from 0 to T.
This ensures that the system remains in the instantaneous ground state throughout the evolution, as long as the evolution is slow enough (according to the adiabatic theorem).
The final state at $t = T$is the ground state of H, which represents the molecular ground state of H_2.
The adiabatic evolution is discretized into multiple small steps. Each step is implemented by a sequence of NMR pulses corresponding to a unitary transformation:

$$U_m^{ad} = e^{-i(\frac{\delta}{2})(1-s_m)\sigma_x} e^{-is_m H\delta} e^{-i(\frac{tau}{2})(1-s_m)\sigma_x} + O(\delta^3)$$

Which can be implemented by pulse sequence $R^s_{-x}(\theta_1) - R^s_{-y}(\theta_2) - R^s_x(\theta_3)$. Here,$\tau$ is the small time increment, and $s_m = \frac{m}{M+1}$where M is the total number of steps, $\theta_1 = -(\frac{\tau}{2})(1 - s_m)$, $\theta_2 = -s_m H\tau$, $\theta_3 = -(\frac{\tau}{2})(1 - s_m)$. In the letter the fidelity of the adiabatic preparation was numerically and experimentally analyzed, they confirmed that longer evolution times lead to higher fidelity, matching the adiabatic theorem.

What is the role of the inital Hamiltonian H_0

The adiabatic theorem states that if a system starts in the ground state of a Hamiltonian H_0 and is slowly evolved to a new Hamiltonian H, it will remain in the ground state throughout, to achieve this, the system is evolved under the time-dependent Hamiltonian, written above.
The choice of H_0 ensures that the evolution from H_0 to H maintains a finite energy gap between the ground state and excited state. This is important because according to the adiabatic theorem, the system will remain in the ground state as long as the evolution is slow enough. The quantum adiabatic theorem states that if a quantum system starts in the ground state of an initial Hamiltonian H_0 and the Hamiltonian is varied slowly enough over time, the system will remain in the instantaneous ground state of the evolving Hamiltonian, provided that There is a nonzero energy gap between the ground state and the first excited state throughout the evolution and the

Hamiltonian changes slowly enough so that transitions to excited states are negligible.
The success of this process depends on the energy gap ΔE between the ground and excited states of $H(s)$, ff the gap is too small, small perturbations or fast changes in s can cause non-adiabatic transitions (jumping to excited states).

Iterative Phase Estimation

The goal of phase estimation is to determine the eigenvalue associated with an eigenstate of a unitary operator. Given the molecular Hamiltonian H, the corresponding time evolution operator is

$$U = e^{-iH\tau},$$

which acts on its eigenstate $|\psi\rangle$ as

$$U|\psi\rangle = e^{i2\pi\phi}|\psi\rangle,$$

where the phase ϕ encodes the energy eigenvalue via

$$E = -\frac{2\pi\phi}{\tau}.$$

Since a full quantum Fourier transform (QFT) would require additional qubits, the experiment employs an iterative approach, extracting ϕ bit by bit.

The process begins by preparing the system qubit (the ^{13}C nucleus) in the ground state of H using adiabatic state preparation (ASP), while the probe qubit (^1H) is initialized in $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. The algorithm then applies a series of controlled-U_k operations, where

$$CU_k = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U_k.$$

The evolution operator is recursively updated as

$$U_{k+1} = (e^{-i2\pi\phi_k} U_k)^{2^n},$$

ensuring that each iteration refines the precision of ϕ. After each controlled operation, the probe qubit's phase is measured using an NMR interferometer, which extracts relative phase shifts from spectral patterns. These measurements provide bits of the binary expansion of ϕ, which are used to update subsequent controlled operations.

Appendix

Program

```
import numpy as np
from qutip import (Qobj, basis, tensor, qeye, sigmax, sigmay, sigmaz)

def product_operator(target, axis, angle):
    """
    Calculate the product operator on a two-qubit system (using QuTiP).
```

```
Parameters:
-----------
target (tuple):
    Tuple containing the indices of target qubits (0, 1, or both).
    Length must be 1 or 2.
axis (str):
    Axis of rotation ('x', 'y', 'z', '-x', '-y', '-z').
angle (float):
    Rotation angle in radians.

Returns:
--------
(qutip.Qobj):
    4x4 Qobj representing the product operator for two qubits.
"""
# Define QuTiP identity and Pauli matrices
I_q = qeye(2)
X_q = sigmax()
Y_q = sigmay()
Z_q = sigmaz()

# Map axis string to QuTiP Pauli operators
axis_to_pauli = {
    'x': X_q,  'y': Y_q,  'z': Z_q,
    '-x': -X_q, '-y': -Y_q, '-z': -Z_q
}

axis_lower = axis.lower()
if axis_lower not in axis_to_pauli:
    raise ValueError("Axis must be one of: 'x', 'y', 'z', '-x', '-y', '-z'")

if not isinstance(target, tuple) or len(target) < 1 or len(target) > 2:
    raise ValueError("Target must be a tuple of length 1 or 2")

for t in target:
    if t not in [0, 1]:
        raise ValueError("Target qubits must be 0 or 1")

pauli_op = axis_to_pauli[axis_lower]

# Construct the generator via tensor products
if len(target) == 1:
    q = target[0]
    if q == 0:
        generator = tensor(pauli_op, I_q)
    else:  # q == 1
        generator = tensor(I_q, pauli_op)
```

```
        else:  # Two-qubit operation
            if target == (0, 1) or target == (1, 0):
                generator = tensor(pauli_op, pauli_op)
            else:
                raise ValueError("For two-qubit operation, target must be (0,1) or (1,0)")

        # Calculate the unitary operation using the matrix exponential
        operator = (-1j * angle / 2 * generator).expm()
        return operator


    def apply_operator_sequence(sequence, initial_state=None, threshold=1e-15):
        """
        Apply a sequence of product operators to a two-qubit system (using QuTiP).

        Parameters:
        -----------
        sequence (list of tuples):
            List of operations in the form [(target, axis, angle), ...]
            where target is a tuple of qubit indices, axis is 'x', 'y', or 'z',
            and angle is the rotation angle in radians.
        initial_state (numpy.ndarray, optional):
            Initial state vector (4x1) or density matrix (4x4).
            If None, starts with |00⟩ state.
        threshold (float, optional):
            Values with absolute magnitude smaller than this threshold
            will be set to zero. Default is 1e-15.

        Returns:
        --------
        final_state (numpy.ndarray):
            Final state after applying all operations (vector or density matrix).
        filtered_count (int):
            Number of values that were set to zero.
        """
        # Define |00> in QuTiP
        ket_0 = basis(2, 0)
        ket_00_qobj = tensor(ket_0, ket_0)  # two-qubit |00>

        # If no initial state given, use |00>
        if initial_state is None:
            # Start as a ket (Qobj)
            state_qobj = ket_00_qobj
        else:
            # Convert the given NumPy array to a QuTiP object
            arr = np.array(initial_state, dtype=complex)
            if arr.shape == (4, 4):
                # Density matrix
```

```
        state_qobj = Qobj(arr, dims=[[2, 2], [2, 2]])
    elif arr.shape == (4,):
        # State vector
        state_qobj = Qobj(arr.reshape((4, 1)), dims=[[2, 2], [1, 1]])
    else:
        raise ValueError("Initial state must be a 4x1 vector or 4x4 density matrix")

# Check if state_qobj is density matrix or ket
is_density_matrix = state_qobj.isoper  # True if operator (density matrix)

# Initialize the net unitary as identity
unitary_net = tensor(qeye(2), qeye(2))  # 4x4 identity

# Build up the net unitary
for op in sequence:
    if len(op) != 3:
        raise ValueError("Each operation must be a tuple of (target, axis, angle)")
    target, axis, angle = op

    operator_qobj = product_operator(target, axis, angle)
    # Accumulate unitary
    unitary_net = operator_qobj * unitary_net

# Apply the net unitary
if is_density_matrix:
    final_qobj = unitary_net * state_qobj * unitary_net.dag()
else:
    final_qobj = unitary_net * state_qobj
# print(unitary_net)

# Convert back to NumPy array to filter small values
final_array = final_qobj.full()

filtered_count = 0
if is_density_matrix:
    # Filter small values in density matrix
    for i in range(4):
        for j in range(4):
            if abs(final_array[i, j]) < threshold:
                final_array[i, j] = 0
                filtered_count += 1
else:
    # Filter small values in state vector
    for i in range(4):
        if abs(final_array[i, 0]) < threshold:
            final_array[i, 0] = 0
            filtered_count += 1
```

```
    # Return final state (shaped as original: 4x1 for ket, 4x4 for density matrix)
    if is_density_matrix:
        final_state = final_array
    else:
        # Flatten back to (4,) if it was a state vector
        final_state = final_array[:, 0]

    return final_state, filtered_count
```

Sequence

Each pulse is discribed by a tuple of three elements: which qubit(s), which axis, what angle.
For example, ((0,), 'x', pi) indecate apply a pi pulse on 0th qubit along x-axis.

```
sequence = [
    [
    ## P1
        ((0,),'x',pi*4),
    ## P1 end

    ## D.J.Pulse
    # H(0)
        ((0,), 'y', pi/4),
        ((0,), 'x', pi),
        ((0,), '-y', pi/4),
    # H(1)
        ((1,), 'y', pi/4),
        ((1,), 'x', pi),
        ((1,), '-y', pi/4),

    ## Oracle
    # x(1) **apply this for f2 and f4 **
        # ((1,),'x',pi),
    # CNOT(0,1) **apply this for f3 and f4 **
        ((0,),'x',pi),
        ((1,), 'y', pi/2),
        ((0, 1), 'z', pi/2),
        ((1,), '-x', pi/2),
        ((0,),'x',pi),

    # H(0)
        ((0,), 'y', pi/4),
        ((0,), 'x', pi),
        ((0,), '-y', pi/4)
    ## D.J. Pulse end
    ],
    [
    ## P2
```

```
# CNOT(0,1)
    ((0,),'x',pi),
    ((1,), 'y', pi/2),
    ((0, 1), 'z', pi/2),
    ((1,), '-x', pi/2),
    ((0,),'x',pi),
# CNOT(1,0)
    ((1,),'x',pi),
    ((0,), 'y', pi/2),
    ((0, 1), 'z', pi/2),
    ((0,), '-x', pi/2),
    ((1,),'x',pi),
## P2 end


## D.J.Pulse
# H(0)
    ((0,), 'y', pi/4),
    ((0,), 'x', pi),
    ((0,), '-y', pi/4),
# H(1)
    ((1,), 'y', pi/4),
    ((1,), 'x', pi),
    ((1,), '-y', pi/4),

## Oracle
# x(1)  **apply this for f2 and f4 **
    # ((1,),'x',pi),
# CNOT(0,1)  **apply this for f3 and f4 **
    ((0,),'x',pi),
    ((1,), 'y', pi/2),
    ((0, 1), 'z', pi/2),
    ((1,), '-x', pi/2),
    ((0,),'x',pi),

# H(0)
    ((0,), 'y', pi/4),
    ((0,), 'x', pi),
    ((0,), '-y', pi/4)
## D.J. Pulse end
],
[
## P3
# CNOT(1,0)
    ((1,),'x',pi),
    ((0,), 'y', pi/2),
    ((0, 1), 'z', pi/2),
    ((0,), '-x', pi/2),
```

```
    ((1,),'x',pi),
# CNOT(0,1)
    ((0,),'x',pi),
    ((1,), 'y', pi/2),
    ((0, 1), 'z', pi/2),
    ((1,), '-x', pi/2),
    ((0,),'x',pi),
## P3 end


## D.J.Pulse
# H(0)
    ((0,), 'y', pi/4),
    ((0,), 'x', pi),
    ((0,), '-y', pi/4),
# H(1)
    ((1,), 'y', pi/4),
    ((1,), 'x', pi),
    ((1,), '-y', pi/4),

## Oracle
# x(1)  **apply this for f2 and f4 **
    # ((1,),'x',pi),
# CNOT(0,1)  **apply this for f3 and f4 **
    ((0,),'x',pi),
    ((1,), 'y', pi/2),
    ((0, 1), 'z', pi/2),
    ((1,), '-x', pi/2),
    ((0,),'x',pi),

# H(0)
    ((0,), 'y', pi/4),
    ((0,), 'x', pi),
    ((0,), '-y', pi/4)
## D.J. Pulse end
]
]
```